LinkedIn AI Agent — Complete Technical Capabilities Report

Project Codename: LinkMind
Version: 1.0.0
Architecture: LangGraph Multi-Agent System + FastAPI Backend + Next.js Frontend
Date: June 27, 2026

Executive Summary

The LinkedIn AI Agent (LinkMind) is a production-grade, autonomous AI system built to manage a user's entire LinkedIn presence — from content drafting to engagement — without requiring the user to manually log into LinkedIn. It operates across three coordinated modes: intelligent content creation, background monitoring with engagement suggestion, and a conversational voice/text chat interface. The system is built on a two-tiered LLM architecture pairing Sarvam-M (a powerful reasoning model) with Groq's Llama-3.3-70b for speed-critical operations, orchestrated through LangGraph stateful agent graphs with PostgreSQL-backed persistence. Human approval gates are built into every action that touches LinkedIn, ensuring the user retains complete control.

1. System Architecture Overview

LinkMind is composed of five integrated layers:

```
┌──────────────────────────────────────────────────────────────────┐
│                    Chat Interface (Next.js 15)                     │
│         Text Chat  │  Voice Input  │  Approval UI  │  Dashboard    │
└──────────────────────────────┬───────────────────────────────────┘
                               │ HTTP / WebSocket / SSE
┌──────────────────────────────▼───────────────────────────────────┐
│                      Backend API (FastAPI)                         │
│   POST /chat  │  /voice/*  │  /pending  │  /approve  │  /monitor   │
└──────┬─────────────────────────────────────────┬─────────────────┘
       │ LangGraph execution                      │ APScheduler
┌──────▼──────────────┐                ┌──────────▼───────────────┐
│  Content Creation   │                │   Monitoring Agent        │
│  Agent (LangGraph)  │                │  (LangGraph + Scheduler)  │
└──────┬──────────────┘                └──────────┬───────────────┘
       │                                          │
┌──────▼──────────────────────────────────────────▼───────────────┐
│              LinkedIn Integration Layer                            │
│  linkedin-api (Voyager) — Reading   │   Playwright Stealth — Posting │
└─────────────────────────────────────┬───────────────────────────┘
                                      │
┌─────────────────────────────────────▼───────────────────────────┐
│              State Store (PostgreSQL)                              │
```

```
│  users │ posts_drafted │ pending_engagements │ watchlist │ chat_history │
└─────────────────────────────────────────────────────────────────────────┘
```

Core Design Principles

- **Human-in-the-loop mandatory:** No action is taken on LinkedIn without explicit user approval via the frontend UI.
- **Stateful orchestration:** LangGraph maintains full graph state across human interrupt/resume cycles, persisted in PostgreSQL.
- **Stealth-first posting:** All write operations to LinkedIn use Playwright with playwright-stealth to simulate realistic human browser behavior with 2–7 second randomized delays between actions.
- **Dual-LLM efficiency:** Heavy reasoning tasks (drafting, evaluation, comment generation) use Sarvam-M; fast classification tasks (intent routing, comment categorization) use Groq Llama-3.3-70b-versatile to minimize latency.
- **Credential security:** All LinkedIn credentials, API keys, and OAuth tokens are stored exclusively in environment variables and encrypted database fields — never exposed in logs, API responses, or client-side code.

2. Component 1: Content Creation Agent

What It Does

The Content Creation Agent is a LangGraph stateful graph that takes a user's raw post idea (text or voice) and transforms it into 2–3 polished, scored LinkedIn post drafts ready for user review and one-click publishing.

Capabilities

- **Multi-draft generation:** Produces 2–3 distinct variations of any post idea. Each variation differs in structure, tone, hook style, and angle — not just surface rewording.
- **Algorithmic scoring:** Each draft is evaluated against LinkedIn best practices and assigned a score out of 100 based on: hook strength (first 3 lines), paragraph structure and readability, hashtag count and relevance (optimal: 3–5), presence of a call-to-action, absence of external links in body text (link-in-comment strategy), estimated reading time, and emotional resonance.
- **Score-weighted ranking:** Drafts are ranked and displayed with color-coded score bars (green ≥80, amber 60–79, red <60).
- **Pre-post editing:** Users can edit any approved draft directly in the UI before final submission.
- **Playwright posting:** After approval, Playwright with stealth mode navigates LinkedIn as a real browser session, types the post with human-like delays, and submits it. The system waits for posting confirmation before updating the draft status to "posted" in PostgreSQL.

- **Failure recovery:** If posting fails, draft status remains "approved" and the error is surfaced to the user with a retry option. Up to 3 retry attempts with exponential backoff are made on all external API calls.

LangGraph Node Flow

```
START
  └─► intake_node           — Parse and normalize user idea + constraints
        └─► style_context_node   — Retrieve past posts from vector store for style co
              └─► draft_node          — Sarvam-M generates 2-3 draft variations
                    └─► evaluate_node      — Score each draft against LinkedIn algori
                          └─► human_approval_node   — INTERRUPT: present drafts to us
                                └─► [user edits/selects]
                                      └─► post_to_linkedin_node   — Playwright posts
                                            └─► log_node            — Persist result 
                                                  └─► END
```

Post Quality Rules (Hard-Coded in Evaluator)

Rule	Threshold	Impact on Score
Hook in first 3 lines	Required	−25 if absent
Hashtag count	3–5	−10 per violation
External link in body	Forbidden	−20 if present
Paragraph length	Max 3 lines each	−5 per long para
CTA at end	Required	−15 if absent
Total length	150–400 words	−10 if outside range
Reading ease	Flesch > 50	−10 if too complex

3. Component 2: Monitoring Agent

What It Does

The Monitoring Agent runs on a scheduled background interval (minimum 2-hour gap, enforced by APScheduler) to scan the user's LinkedIn posts for new engagement and fetch recent posts from all profiles in the user's Watchlist. It then uses Sarvam-M to generate contextually appropriate suggested reactions and comments, which are queued for user review in the Pending Queue.

Capabilities

- **Self-post monitoring:** Retrieves new comments and reactions on the authenticated user's recent posts via the linkedin-api Voyager library. Classifies each comment as: question, agreement, criticism, spam, or opportunity.
- **Watchlist monitoring:** For each profile in the Watchlist, fetches their latest posts. Analyzes post content to determine if the user's expertise is relevant and if engagement would be beneficial.
- **Intelligent comment generation:** For each engagement opportunity, Sarvam-M generates a contextually appropriate comment that reflects the user's voice, expertise (ML, FPGA, hardware, distributed systems), and tone. Comments are not generic — they reference specific content from the post being replied to.
- **Reaction suggestion:** For posts where a full comment isn't warranted, the agent suggests the appropriate reaction type (Like, Insightful, Celebrate, Support).
- **Rate limit compliance:** The 2-hour minimum monitoring interval is hard-enforced. The agent tracks its own request cadence and backs off if LinkedIn signals throttling behavior.
- **Pending Queue management:** All generated suggestions are stored in PostgreSQL with status "pending". They persist across server restarts and are accessible from any device.

Scheduling Architecture

```
# APScheduler configuration
scheduler = BackgroundScheduler()
scheduler.add_job(
    monitoring_agent.run,
    trigger=IntervalTrigger(hours=2),
    id='linkedin_monitor',
    replace_existing=True,
    misfire_grace_time=300  # 5-minute grace window
)
```

Engagement Decision Logic

```
For each fetched item:
  |
  ├─ Is it from a Watchlist profile?
  |      └─ Is the topic relevant to user's expertise? → Generate comment suggestion
  |      └─ Is the topic irrelevant? → Generate reaction suggestion
  |
  └─ Is it a comment on user's own post?
         └─ Is it a question? → Generate answer comment
         └─ Is it positive feedback? → Generate thank-you comment
         └─ Is it criticism? → Flag for manual review, no auto-suggest
         └─ Is it spam? → Suggest no action, auto-skip
```

4. Component 3: Chat Interface & Voice System

Chat Interface Capabilities

The Chat_Interface is a Next.js 15 (App Router) web application that serves as the single control surface for the entire agent system. It supports both text and voice interaction.

Text chat:

- Natural language commands are routed to the correct agent via Groq Llama-3.3-70b intent classification (Fast_LLM).
- Responses stream token-by-token via Server-Sent Events (SSE) from FastAPI — no waiting for complete responses.
- Draft cards, score bars, pending action cards, and watchlist management are all surfaced as rich UI components inline in the chat thread.
- Full conversation history is persisted in PostgreSQL and restored on page reload.

Supported intent types and routing:

User Intent	Classified As	Routed To
"Write a post about X"	`create_post`	Content_Creation_Agent
"Show me pending actions"	`view_pending`	Pending Queue endpoint
"Add @username to my watchlist"	`add_watchlist`	Watchlist management
"What is [general question]?"	`general_query`	Sarvam-M direct response
"Post this draft"	`approve_action`	LinkedIn_Poster via LangGraph resume

Voice Input Capabilities (Sarvam STT)

- **Service:** Sarvam Saarika speech-to-text model via Sarvam AI API.
- **Languages:** English, Hindi, Hinglish (code-switched) — all supported natively.
- **Modes:** Real-time streaming via WebSocket for live transcription; batch REST for recorded audio.
- **Frontend:** Web Audio API AnalyserNode drives a real-time 32-bar waveform visualization while recording.
- **Error handling:** If transcription fails (network error, unclear audio), a descriptive error is shown and the user can retry or switch to text input.

Voice Output Capabilities (Sarvam TTS)

- **Service:** Sarvam Bulbul v3 text-to-speech model.
- **Languages:** English, Hindi, Hinglish — agent can respond in the same language the user spoke in.
- **Opt-in:** Voice output is disabled by default. Users toggle it on from the interface.

- **Graceful fallback:** If TTS generation fails, the text response is displayed normally without interrupting the conversation flow.

5. Component 4: LinkedIn Integration Layer

Reader (linkedin-api Voyager)

The LinkedIn_Reader uses the unofficial Voyager API (the same internal API LinkedIn's own web client uses) for all read operations. This avoids the restrictive official API tier limitations.

Read capabilities:

- Fetch authenticated user's profile and Member_ID
- Retrieve user's recent posts (up to 20 most recent)
- Retrieve comments and reactions on any post by URN
- Fetch recent posts from any LinkedIn profile by Member_ID or profile URL
- Validate whether a LinkedIn profile exists (for Watchlist add validation)

Authentication: Cookie-based session authentication using stored `li_at` cookie from environment variables. Session is validated on startup and before each operation.

Poster (Playwright with Stealth)

The LinkedIn_Poster uses Playwright (headless Chromium) with the `playwright-stealth` plugin to simulate a real human browser session for all write operations.

Write capabilities:

- Create new LinkedIn posts (text-only in v1; media support planned for v2)
- Post comments on LinkedIn posts
- React to LinkedIn posts with specified reaction types

Anti-detection measures:

- playwright-stealth masks all Playwright fingerprints (navigator.webdriver, Chrome runtime, etc.)
- Randomized Human_Like_Delay of 2–7 seconds applied before every action
- Realistic typing simulation — characters typed at randomized intervals, not pasted instantly
- Browser viewport and user-agent spoofed to match a real desktop Chrome session
- No headless mode indicators in navigator properties

6. Component 5: Watchlist Management

Capabilities

- **Add profile:** User provides a LinkedIn profile URL or Member_ID. The system validates the profile exists via LinkedIn_Reader, then stores it in the `watchlist` PostgreSQL table with profile metadata (name, headline, last checked timestamp).
- **Remove profile:** Immediate removal from watchlist; next monitoring cycle will not include the profile.
- **View watchlist:** Returns all profiles with their metadata and last monitoring timestamp.
- **Monitoring integration:** Every scheduled Monitoring_Agent run automatically iterates through the full Watchlist and fetches recent posts from each profile.

7. LLM Architecture — Two-Tiered Design

Primary LLM: Sarvam-M

Property	Value
Provider	Sarvam AI
Endpoint	`https://api.sarvam.ai/v1` (OpenAI-compatible)
Use cases	Post drafting, content evaluation, comment generation, conversational responses, engagement opportunity analysis
Why	Strong reasoning, Indian language awareness, suitable for nuanced LinkedIn content for Indian professional audience

Fast LLM: Groq Llama-3.3-70b-versatile

Property	Value
Provider	Groq (hardware-accelerated inference)
Use cases	Intent classification, quick comment categorization, routing decisions, parsing voice transcripts
Why	Sub-100ms inference for tasks that only need classification — keeps the UI feeling instant

Retry Policy (Both LLMs)

```
Attempt 1 → fail → wait 1s
Attempt 2 → fail → wait 2s
Attempt 3 → fail → return error to user with descriptive message
```

8. Database Schema

All data is persisted in PostgreSQL. The schema supports full LangGraph state resumption across server restarts.

Tables

`users`

```
id             UUID PRIMARY KEY
linkedin_email TEXT NOT NULL
member_id      TEXT NOT NULL           -- LinkedIn internal URN
access_token   TEXT                    -- OAuth token (if API path)
li_at_cookie   TEXT                    -- Voyager cookie (if cookie path)
created_at     TIMESTAMPTZ DEFAULT NOW()
```

`posts_drafted`

```
id             UUID PRIMARY KEY
user_id        UUID REFERENCES users(id)
content        TEXT NOT NULL
score          INTEGER                 -- 0-100 evaluation score
status         TEXT                    -- draft | approved | posted | failed
langgraph_thread_id TEXT              -- for state resumption
created_at     TIMESTAMPTZ DEFAULT NOW()
posted_at      TIMESTAMPTZ
```

`pending_engagements`

```
id             UUID PRIMARY KEY
user_id        UUID REFERENCES users(id)
type           TEXT                    -- comment | reaction
target_post_urn TEXT NOT NULL
suggested_content TEXT
status         TEXT                    -- pending | approved | skipped | executed
langgraph_thread_id TEXT
created_at     TIMESTAMPTZ DEFAULT NOW()
```

`watchlist`

```
id             UUID PRIMARY KEY
user_id        UUID REFERENCES users(id)
linkedin_member_id TEXT NOT NULL
display_name   TEXT
headline       TEXT
last_checked   TIMESTAMPTZ
added_at       TIMESTAMPTZ DEFAULT NOW()
```

`chat_history`

```
id             UUID PRIMARY KEY
user_id         UUID REFERENCES users(id)
role            TEXT                    -- user | assistant
content         TEXT NOT NULL
intent          TEXT                    -- classified intent
created_at      TIMESTAMPTZ DEFAULT NOW()
```

9. API Endpoints

Method	Endpoint	Description
POST	`/chat`	Send message; streams response via SSE
POST	`/voice/transcribe`	Audio blob → text via Sarvam STT
POST	`/voice/speak`	Text → audio via Sarvam TTS Bulbul v3
GET	`/pending`	Get all pending engagement actions
POST	`/approve/{action_id}`	Approve and execute an engagement action
DELETE	`/skip/{action_id}`	Skip/remove a pending action
POST	`/monitor/add`	Add a LinkedIn profile to Watchlist
GET	`/monitor/watchlist`	Get current Watchlist

All endpoints return 400 with descriptive messages on invalid input. All endpoints return 500 with structured error objects on internal failures (never raw stack traces to the client).

10. Security Model

- **No credentials in code:** All LinkedIn credentials, API keys (Sarvam, Groq), and OAuth tokens live exclusively in `.env` files loaded at process start.
- **Startup validation:** FastAPI startup event checks all required environment variables are present before accepting traffic. If any are missing, the server refuses to start with a clear error listing the missing keys.
- **No credential logging:** Structured logging middleware explicitly strips any field containing `token`, `password`, `cookie`, `secret`, or `key` before writing log entries.
- **HTTPS only:** All external API communications (Sarvam, Groq, LinkedIn OAuth) are made exclusively over HTTPS.
- **Rate limit respect:** The 2-hour monitoring interval floor is architectural — it cannot be bypassed through the API. LinkedIn session rate limits are monitored, and the agent backs off automatically.

11. Error Handling and Logging

- **Structured logging** with severity levels: DEBUG, INFO, WARNING, ERROR.
- Every log entry includes: timestamp (ISO 8601), component name (e.g., `content_creation_agent`, `linkedin_poster`), operation name, and stack trace on ERROR.
- **External API failures** log full request context (endpoint, method, status code) without logging request bodies that might contain credentials.
- **User-facing errors** are always friendly strings — never raw Python exceptions.
- **Critical errors** (LinkedIn session expired, all LLM retries exhausted) are surfaced immediately via the Chat_Interface as inline error messages.

12. Technology Stack Summary

Layer	Technology	Version
Frontend framework	Next.js (App Router)	15
Frontend language	TypeScript	5.x
Frontend styling	Tailwind CSS	v4
Animation — page/timeline	GSAP (GreenSock)	3.x
Animation — React components	Motion for React	11.x
Animation — micro-interactions	Anime.js	v4
Animation — onboarding video	HyperFrames by HeyGen	latest
State management	Zustand	5.x
Server state	TanStack Query	v5
Backend framework	FastAPI	0.115.x
Agent orchestration	LangGraph	0.2.x
Scheduler	APScheduler	3.x
Primary LLM	Sarvam-M	via api.sarvam.ai/v1
Fast LLM	Groq Llama-3.3-70b-versatile	via api.groq.com
STT	Sarvam Saarika	via Sarvam AI
TTS	Sarvam Bulbul v3	via Sarvam AI
LinkedIn reader	linkedin-api (Voyager)	2.x
LinkedIn poster	Playwright + playwright-stealth	1.4x
Database	PostgreSQL	16
Language (backend)	Python	3.11+

13. Known Limitations (v1.0)

- **Media posts not supported in v1:** Only text posts can be created and published. Image/video/document posts are planned for v2.
- **Single user only in v1:** The system is designed for one authenticated LinkedIn account. Multi-user support requires OAuth flow and per-user token management.
- **Unofficial Voyager API risk:** linkedin-api uses LinkedIn's internal Voyager API, which is unofficial and can break if LinkedIn changes their internal API schema.
- **Playwright session management:** Browser sessions can expire if not refreshed; the system detects auth failures and surfaces a re-login prompt but does not auto-refresh Playwright sessions.
- **No A/B post testing:** The scoring system evaluates drafts algorithmically but does not have historical performance data to calibrate scores against actual engagement metrics (planned for v2 with post analytics integration).

14. Planned v2 Features

- Media post support (images, PDFs, carousels via LinkedIn Assets API)
- Multi-user OAuth with per-user token management
- Post scheduling — queue approved posts for optimal posting times
- Analytics dashboard — track actual post performance (views, likes, comments) vs predicted score
- Style learning — fine-tune prompt conditioning on user's highest-performing historical posts
- Company page support — post on behalf of LinkedIn company pages (requires Community Management API product)
- Auto-refresh of Playwright browser sessions